

# Repository Structure

---

Complete overview of the RAG tutorial repository.

## ■ Directory Layout

---

```
rag_tutorial/
├──
│   ├── README.md                # Main introduction and navigation
│   ├── QUICKSTART.md           # 5-minute setup guide
│   ├── requirements.txt         # Python dependencies
│   ├── .env.example             # Environment variables template
│   └── demo.py                  # Quick demonstration script
├──
│   └── lessons/                 # Step-by-step tutorials
│       ├── 01-introduction-to-rag.md
│       ├── 02-understanding-embeddings.md
│       ├── 03-vector-databases-retrieval.md
│       ├── 04-language-models-generation.md
│       ├── 05-building-simple-rag.md
│       └── 06-advanced-rag-techniques.md
├──
│   └── examples/                # Working code examples
│       ├── 01-simple-qa/
│       │   ├── simple_qa.py
│       │   └── README.md
│       └── 02-document-chat/
│           ├── document_chat.py
│           └── README.md
├──
│   └── utils/                   # Reusable utility functions
│       ├── __init__.py
│       ├── embeddings.py        # Embedding operations
│       ├── retrieval.py         # Vector search
│       └── generation.py        # LLM generation
├──
│   └── data/                    # Sample data for testing
│       └── sample_rag_article.txt
```

## ■ Learning Path

---

For Complete Beginners

1. **Start Here:** Read [README.md](#) for overview
2. **Quick Setup:** Follow [QUICKSTART.md](#)
3. **Run Demo:** Execute `python demo.py` to see RAG in action
4. **Learn Concepts:** Read lessons 1-4 in order
5. **Build System:** Follow lesson 5 to build your first RAG system
6. **Try Example:** Run `examples/01-simple-qa/simple_qa.py`
7. **Go Advanced:** Study lesson 6 for production techniques

## For Experienced Developers

1. **Skim:** [README.md](#) and [QUICKSTART.md](#)
2. **Setup:** `pip install -r requirements.txt`
3. **Review:** Lessons 3-4 for implementation details
4. **Code:** Study `examples/` and `utils/` code
5. **Advanced:** Jump to lesson 6 for techniques
6. **Build:** Create your own system using utilities

# ■ Lesson Summaries

---

## Lesson 1: Introduction to RAG

- What RAG is and why it matters
- Problems it solves (hallucinations, knowledge cutoff)
- Three-step process: Index, Retrieve, Generate
- Real-world applications

## Lesson 2: Understanding Embeddings

- What embeddings are (numerical text representations)
- How they capture semantic meaning
- Popular embedding models
- Cosine similarity for measuring relevance

## Lesson 3: Vector Databases & Retrieval

- Why specialized databases are needed

- Approximate nearest neighbor (ANN) algorithms
- Popular vector databases (Chroma, Pinecone, etc.)
- Document chunking strategies
- Retrieval techniques

## **Lesson 4: Language Models & Generation**

- How LLMs work at a high level
- Effective prompting for RAG
- Choosing and using LLMs
- Handling common issues

## **Lesson 5: Building a Simple RAG System**

- Complete implementation walkthrough
- Document loading and chunking
- Building the RAG pipeline

- Command-line interface
- Testing and evaluation

## Lesson 6: Advanced RAG Techniques

- Semantic chunking
- Hybrid search (semantic + keyword)
- Re-ranking for better results
- Query expansion and multi-query
- Production optimizations

## ■ Example Summaries

---

### Example 1: Simple QA

**File:** `examples/01-simple-qa/simple_qa.py`

Minimal RAG system that: - Loads facts into vector database - Answers questions using retrieval - Works without OpenAI API key - Perfect for understanding basics

**Run:** `python examples/01-simple-qa/simple_qa.py`

## Example 2: Document Chat

**File:** `examples/02-document-chat/document_chat.py`

Interactive chat system that: - Loads text documents - Chunks them intelligently - Maintains conversation history - Cites sources in answers - Requires OpenAI API key

**Run:** `python examples/02-document-chat/document_chat.py`

## ■ ■ Utility Modules

---

`utils/embeddings.py`

- `EmbeddingManager`: Manage embedding models
- `embed_text()`: Convert text to vectors
- `compute_similarity()`: Calculate text similarity

**Example:**

```
from utils import EmbeddingManager
embedder = EmbeddingManager()
vector = embedder.embed("Hello world")
```

## utils/retrieval.py

- **Retriever**: Basic vector search
- **HybridRetriever**: Advanced retrieval with filtering
- Search with thresholds and MMR

### Example:

```
from utils import Retriever
retriever = Retriever()
retriever.add_documents(["doc1", "doc2"])
results = retriever.search("query")
```

## utils/generation.py

- **Generator**: LLM text generation
- **PromptTemplate**: Reusable prompt templates
- RAG-specific generation helpers

### Example:

```
from utils import Generator
generator = Generator(api_key="...")
answer = generator.generate_with_context(query, context)
```

## ■ Sample Data

---

`data/sample_rag_article.txt`

Comprehensive article about RAG covering: - Introduction and motivation - How RAG works - Key components - Best practices - Applications - Challenges

Use this for testing your RAG systems!

## ■ Quick Start Commands

---

```
# Setup
source .venv/bin/activate
pip install -r requirements.txt

# Run demo
python demo.py

# Run example 1 (no API key)
python examples/01-simple-qa/simple_qa.py

# Run example 2 (needs API key)
export OPENAI_API_KEY="your-key"
python examples/02-document-chat/document_chat.py

# Test utilities
python utils/embeddings.py
python utils/retrieval.py
```

## ■ File Purposes

---

| File                          | Purpose                    |
|-------------------------------|----------------------------|
| <code>README.md</code>        | Main entry point, overview |
| <code>QUICKSTART.md</code>    | Fast setup guide           |
| <code>requirements.txt</code> | Python dependencies        |



|                           |                               |
|---------------------------|-------------------------------|
| <code>.env.example</code> | Environment variable template |
| <code>demo.py</code>      | Quick demonstration           |
| <code>lessons/*.md</code> | Educational content           |
| <code>examples/*/</code>  | Working code samples          |
| <code>utils/*.py</code>   | Reusable utilities            |
| <code>data/*.txt</code>   | Sample documents              |

## ■ Where to Go Next

---

- **New to RAG?** → Start with [Lesson 1](#)
- **Want to code?** → Run `demo.py` and examples
- **Need utilities?** → Check `utils/` modules
- **Building app?** → Study [Lesson 5](#)
- **Production ready?** → Read [Lesson 6](#)

---

Happy Learning! ■